FOUNDATIONS OF CONVOLUTIONAL NEURAL NETWORKS

Goal: Implement the foundational layers of CNNs (pooling, convolutions) and stack them properly in a deep network to solve multi-class image classification problems.

Learning Objectives:

- Explain the convolution operation
- Apply two different types of pooling operations
- Identify the components used in a convolutional neural network (padding, stride, filter...) and their purpose.
- Build a CNN.



deeplearning.ai

Convolutional Neural Networks

Computer vision

Applications of deep learning in computer vision:

- Self-driving cars: detect other cars and pedestrians around.
- Face recognition
- Art creation

⋮

Computer Vision Problems

Image Classification



64x64

→ Cat? (0/1)

Object detection



- Detect the position of other cars around you.

Neural Style Transfer

Content image



Style image



Picasso



Paint the "content image" using the style of the "style image"

Andrew Ng

Deep Learning on large images



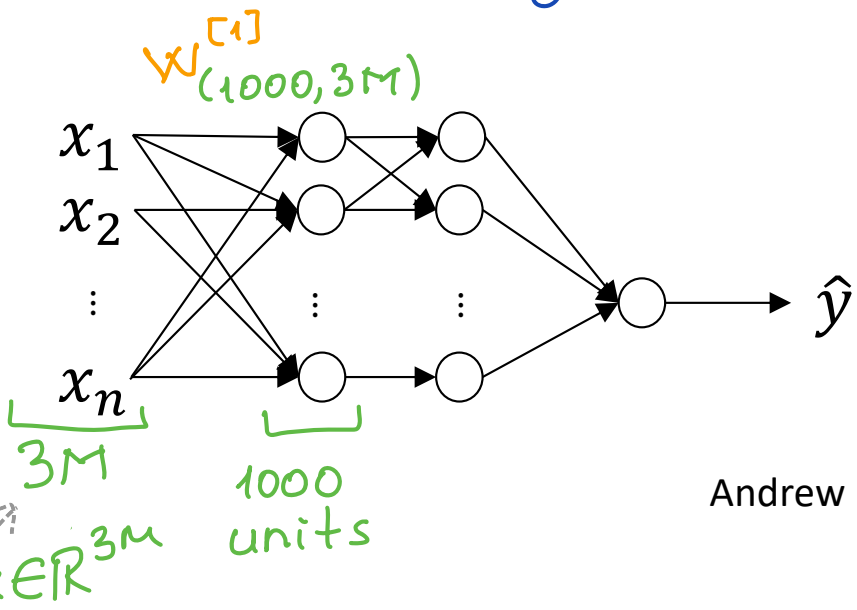
64x64

Cat? (0/1)

$64 \times 64 \times 3 = 12288$ dimensions (not very big)



$1000 \times 1000 \times 3 = 3$ million features



Andrew Ng

- $W^{[1]}$ will have $1000 \times 3M = 3$ billion parameters!! (very large)
- It is difficult to get enough data to prevent overfitting.
- Computational requirements and memory requirements are large.
- We want to process large images in computer vision.
- We need the convolution operation.



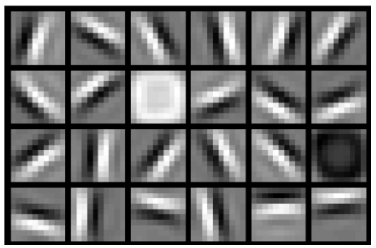
deeplearning.ai

Convolutional Neural Networks

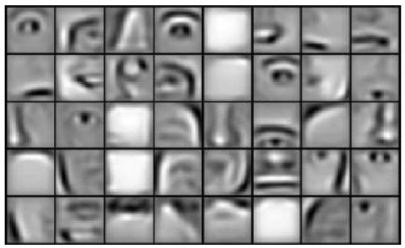
Edge detection example

Computer Vision Problem

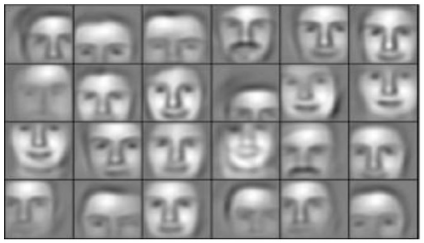
later layers detect complete objects (faces)

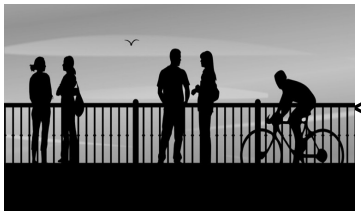


Earlier layers detect edges



later layers detect parts of objects

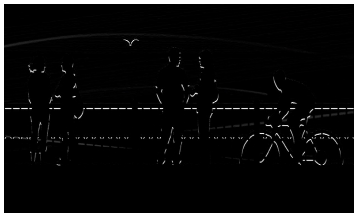




input image



vertical edges



horizontal edges

- goal is to detect objects
- first step: detect edges

Andrew Ng

Vertical edge detection

image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

convolution operation

*

1	0	-1
1	0	-1
1	0	-1

3×3

filter

- We will convolve the image and the filter.

Andrew Ng

Vertical edge detection

paste the filter here

3^{1}	0^{0}	1^{-1}	2	7	4
1^{1}	5^{0}	8^{-1}	9	3	1
2^{1}	7^{0}	2^{-1}	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

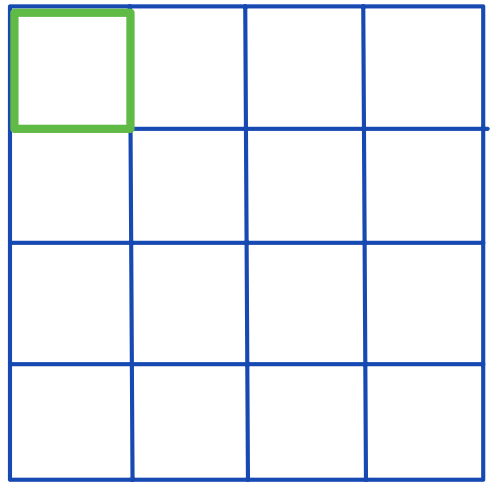
*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

want to compute this element



4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add:

3^{1}	0^{0}	1^{-1}	2	7	4
1^{1}	5^{0}	8^{-1}	9	3	1
2^{1}	7^{0}	2^{-1}	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

$3\times1+1\times1+2\times1+$
$0\times0+5\times0+7\times0+$
$1\times-1+8\times-1+2\times-1=-5$

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

want to compute this element

-5			

4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add:

3	0^{1}	1^{0}	2^{-1}	7	4
1	5^{1}	8^{0}	9^{-1}	3	1
2	7^{1}	2^{0}	5^{-1}	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

want to compute this element

-5	-4		

4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add

3	0	1^{1}	2^{0}	7^{-1}	4
1	5	8^{1}	9^{0}	3^{-1}	1
2	7	2^{1}	5^{0}	1^{-1}	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

want to compute this element

-5	-4	0	

4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add

3	0	1	2^{1}	7^{0}	4^{-1}
1	5	8	9^{1}	3^{0}	1^{-1}
2	7	2	5^{1}	1^{0}	3^{-1}
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

want to compute this element →

-5	-4	0	8

4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add

3	0	1	2	7	4
1^{1}	5^{0}	8^{-1}	9	3	1
2^{1}	7^{0}	2^{-1}	5	1	3
0^{1}	1^{0}	3^{-1}	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

$*$

1	0	-1
1	0	-1
1	0	-1

3×3 filter

$=$

want to compute this element

-5	-4	0	8
-10			

4×4

Andrew Ng

Vertical edge detection

paste the filter here → multiply and add

3	0	1	2	7	4
1	5^{1}	8^{0}	9^{-1}	3	1
2	7^{1}	2^{0}	5^{-1}	1	3
0	1^{1}	3^{0}	1^{-1}	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

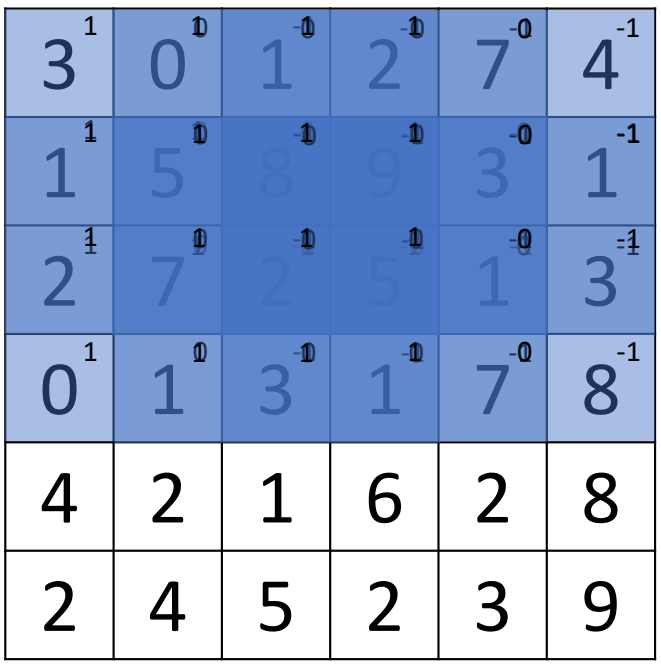
want to compute this element

-5	-4	0	8
-10	-2		

4×4

Andrew Ng

Vertical edge detection



6×6

*

1	0	-1
1	0	-1
1	0	-1

3×3 filter

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4×4

Implementation:

Python : conv_forward

Tensorflow : tf.nn.conv2d

Keras : Conv2D

Andrew Ng

Vertical edge detection

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6x6

*

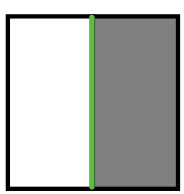
1	0	-1
1	0	-1
1	0	-1

3x3

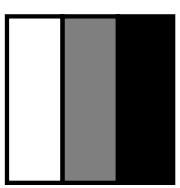
=

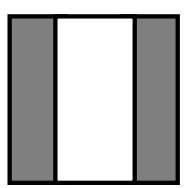
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4x4



*





brighter pixels

vertical edge

darker pixels

strong edge in the middle of the image is detected

Andrew Ng



deeplearning.ai

Convolutional Neural Networks

More edge detection

- Goal: Learn the difference between positive and negative edges.
  (Difference btwn light to dark and dark to light transitions.)

Vertical edge detection examples

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1



=

vertical edge is detected

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0





(light to dark)

(positive edge)

0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0





(dark to light) (negative edge)

Andrew Ng



dark to light edges are indicated by -30.

- If the transition from light to dark (or vice versa) is not important, we can take the absolute value.

Vertical and Horizontal Edge Detection

1	0	-1
1	0	-1
1	0	-1

Vertical edge detector

1	1	1
0	0	0
-1	-1	-1

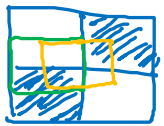
Horizontal edge detector

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

6 x 6

*

1	1	1
0	0	0
-1	-1	-1



=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

intermediate value:
filter captures part of the positive edge on the left and part of the negative edge on the right.

- If this was a very large (e.g. 1000×1000) image, then the intermediate values would be quite small relative to the size of the image.

Learning to detect edges: other options

1	0	-1
1	0	-1
1	0	-1

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

convolution

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

3×3

=

45°
70°
73°
→ edge orientations

- Advantage of Sobel filter: it gives more weight to the central row and the central pixel, which makes the filter more robust.

* In deep learning, we don't have to hand-pick the nine filter coefficients. We can learn just learn them and treat the nine numbers as parameters, which can be learned using backpropagation. (parameters: $w_1, w_2, \dots w_9$)
- It can learn a Sobel filter, Scharr filter or another filter which is better at capturing statistics of your data.

- Convolution operation allows back propagation to learn whatever 3x3 filter it wants.



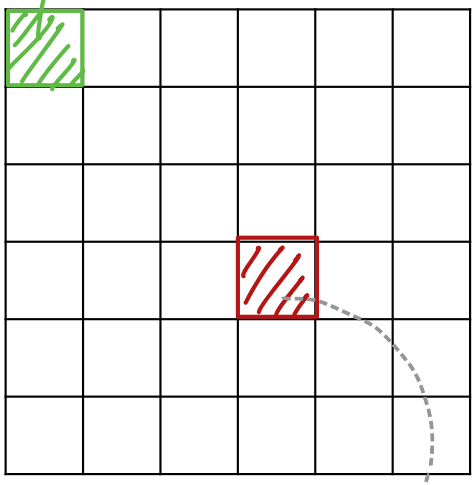
deeplearning.ai

Convolutional Neural Networks

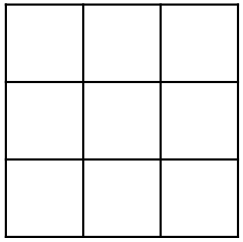
Padding

A modification to the basic convolutional operation to prevent the output image from shrinking.

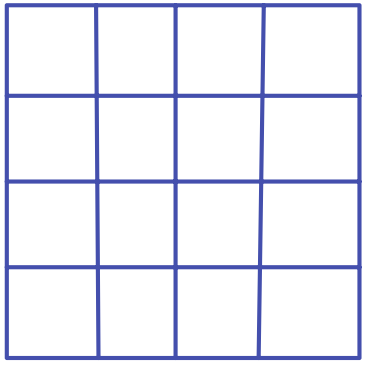
Padding



*



=



only used in the computation of this output.

Since there are only 4x4 possible positions where the 3x3 filter completely fits in the 6x6 image

Andrew Ng

- General rule for computation of size:

  Size of image: $n \times n$, 6×6

  Size of filter: $f \times f$, 3×3

  Size of convolution of image and filter: $(n-f+1) \times (n-f+1)$

  $(6-3+1) \times (6-3+1)$

  4×4

- Down sides:

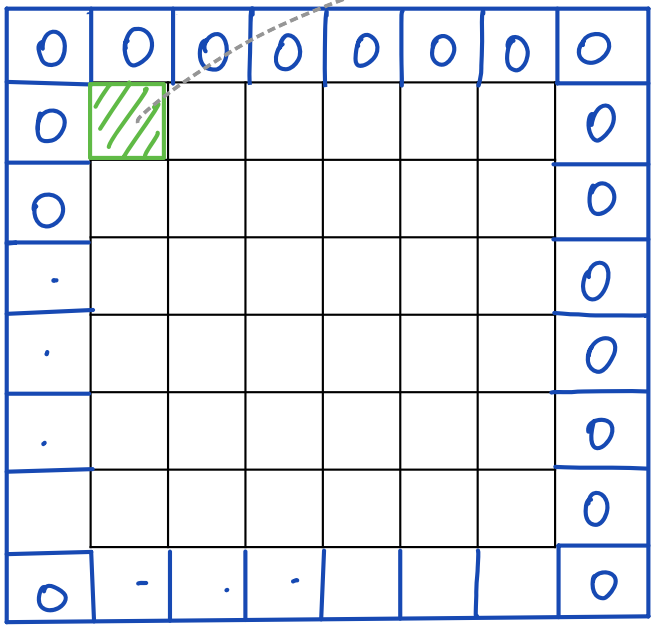
  1) Every time the convolution operator is applied, the image shrinks. We don't want the image to shrink every time we detect edges or other features. After many layers in a deep network, the image will be very small.

  2) The green pixel at the corner of the image is touched by the filter only once. (A pixel in the middle is used 9 times by the 3×3 filter). So, pixels in the corners are are used used less in the output. We're throwing away a lot of the information near the edge of the image.

- Solution to both problems above: pad the image before convolution.

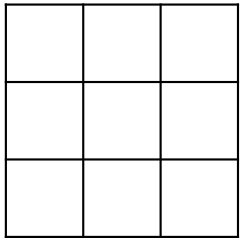
Padding

The green pixel in the corner influences the green pixels in the output, after padding.



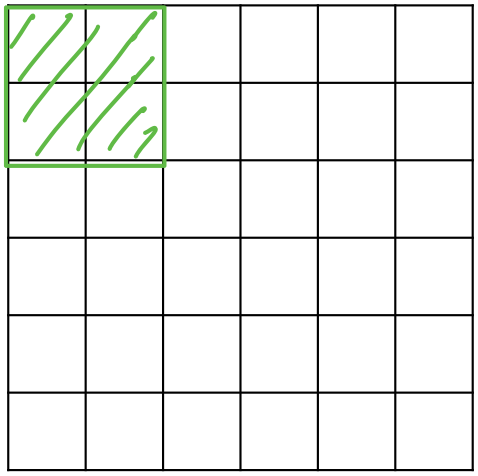
8x8 input image

*



3x3

=



6x6 output image

Pad the image by 1 pixel of zeros (by convention) around all borders. (padding = 1)

output image size = $(n+2p-f+1) \times (n+2p-f+1)$

$(6+2-3+1) \times (6+2-3+1)$

6×6

Andrew Ng

- After padding the green corner pixel influencer a 4x4 area in the output. The second problem above is reduced.

- The image can also be padded by 2 or more pixels around the borders.

- How much padding should we use? There are two common methods: "valid" and "same".
    - Valid : no padding

Valid and Same convolutions

"Valid": No padding

$n \times n$ (input) $*$ $f \times f$ (filter) $\rightarrow (n-f+1) \times (n-f+1)$ (output)

6×6 $*$ 3×3 $\rightarrow$ 4×4

"Same": Pad so that output size is the same as the input size.

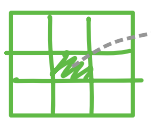
output size: $(n+2p-f+1) \times (n+2p-f+1)$

we want: $n+2p-f+1 = n \Rightarrow \boxed{p = \frac{f-1}{2}}$ for output size = input size

e.g. 3×3 filter $\Rightarrow p = \frac{3-1}{2} = 1$

5×3 filter $\Rightarrow p = \frac{5-1}{2} = 2$

(f is usually odd: 1×1, 3×3, 5×5, 7×7)



center pixel

Andrew Ng



deeplearning.ai

Convolutional Neural Networks

Strided convolutions

Strided convolution

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

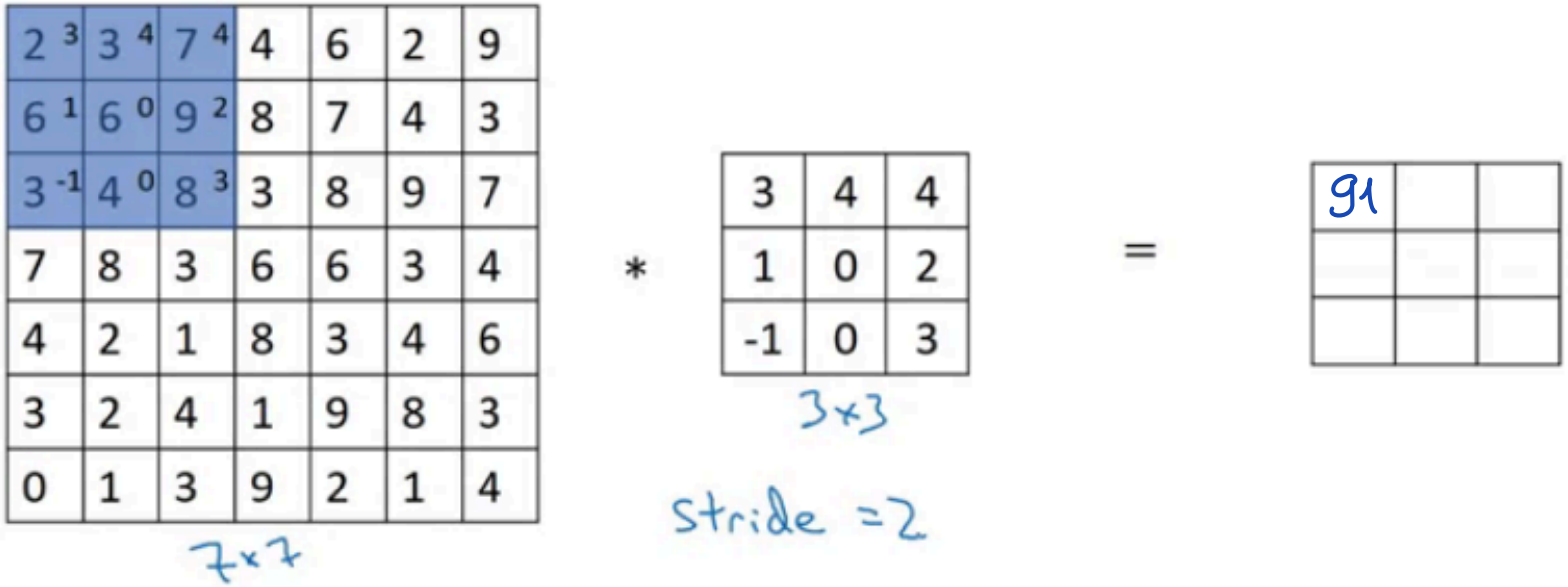
*

3	4	4
1	0	2
-1	0	3

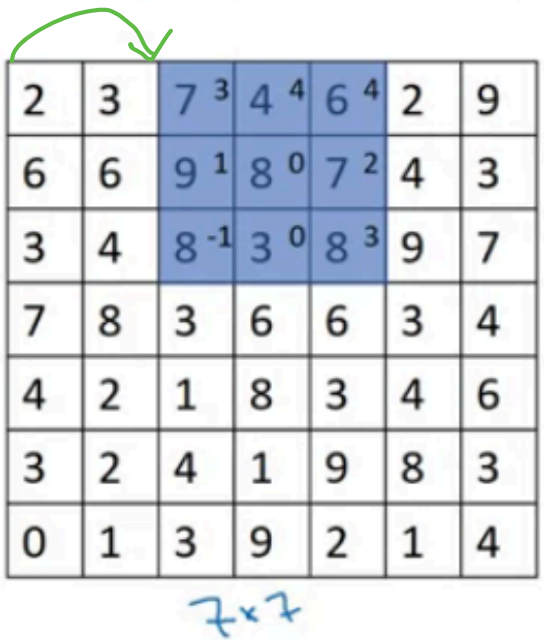
3×3

=

Strided convolution



Strided convolution



*

3	4	4
1	0	2
-1	0	3

3×3

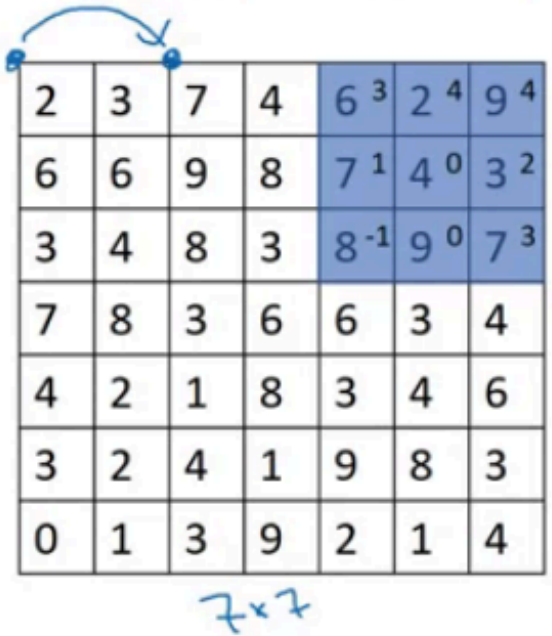
=

91	100	

Stride = 2

- Take two steps horizontally instead of 1 step.

Strided convolution



*

3	4	4
1	0	2
-1	0	3

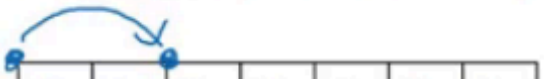
3×3

Stride = 2

=

91	100	83

Strided convolution



2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

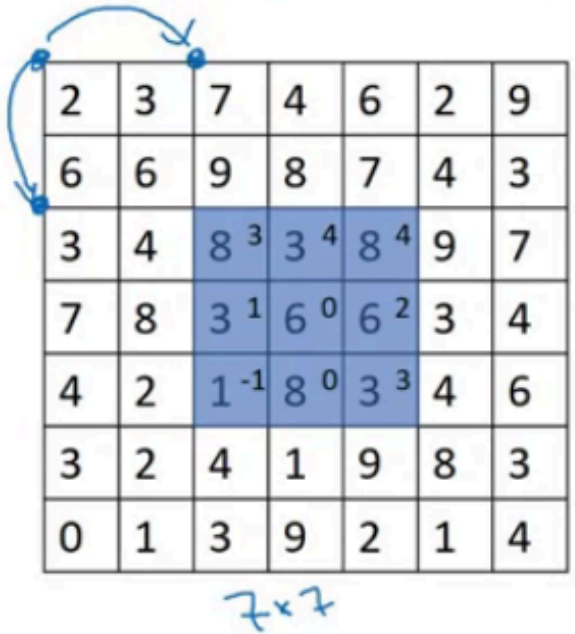
stride = 2

=

91	100	83
69		

Andrew Ng

Strided convolution



*

3	4	4
1	0	2
-1	0	3

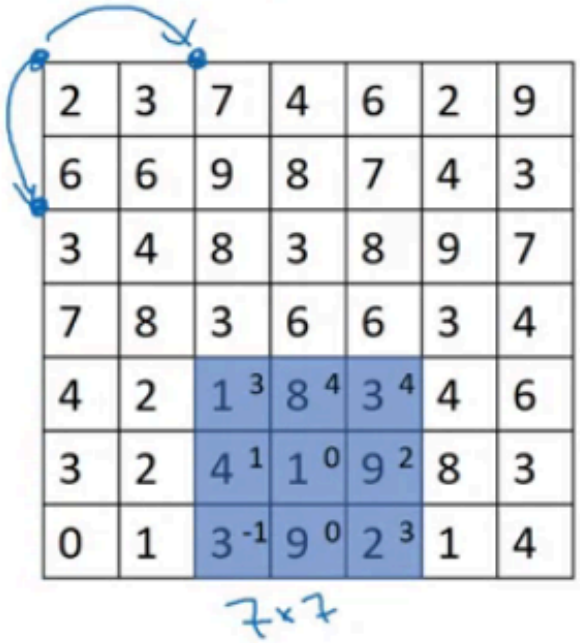
3×3

stride = 2

=

91	100	83
69	91	

Strided convolution



2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1 3	8 4	3 4	4	6
3	2	4 1	1 0	9 2	8	3
0	1	3 -1	9 0	2 3	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

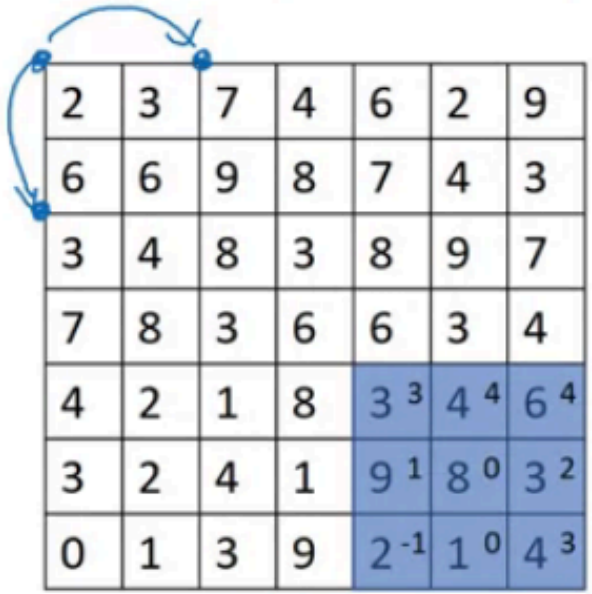
3×3

stride = 2

=

91	100	83
69	91	127
44	72	

Strided convolution



2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3^{3}	4^{4}	6^{4}
3	2	4	1	9^{1}	8^{0}	3^{2}
0	1	3	9	2^{-1}	1^{0}	4^{3}

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

stride = 2

=

91	100	83
69	91	127
44	72	74

3×3

Formula for output size: $n \times n$ (input) $*$ $f \times f$ (filter)
padding p, strides s

output size: $\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$ → floor (round down to

Andrew Ng

- If $n=7$, $p=0$, $s=2$, $f=3$

  output size: $\left\lfloor \frac{7+0-3}{2} + 1 \right\rfloor = 3 \Rightarrow 3 \times 3$

- The filter must lie entirely within the image or the image plus the padding region to generate the corresponding output pixel. This is the convention. So, we need to round down.

Summary of convolutions

$n \times n$ image $\quad$ $f \times f$ filter

padding p $\quad$ stride s

Output size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

Technical note on cross-correlation vs. convolution

Convolution in math textbook:

2	3	7	4	6	2
6	6	9	8	7	4
3	4	8	3	8	9
7	8	3	6	6	3
4	2	1	8	3	4
3	2	4	1	9	8

*

3	4	5
1	0	2
-1	9	7

- In previous slides, it was actually "cross-correlation" that was carried out.
- But in DL it is called "convolution" by convention.

Before convolution flip the filter both vertically and horizontally:

7	9	-1
2	0	1
5	4	3

Apply convolution afterwards.

⊛ If you double flip the filter before convolution the following associativity property holds:

$$(A * B) * C = A * (B * C)$$

⊛ But in DL omitting the flipping operation simplifies the code and the NN works just as well.

⊛ In ML this is called as "convolution", although mathematically it is "cross-correlation".



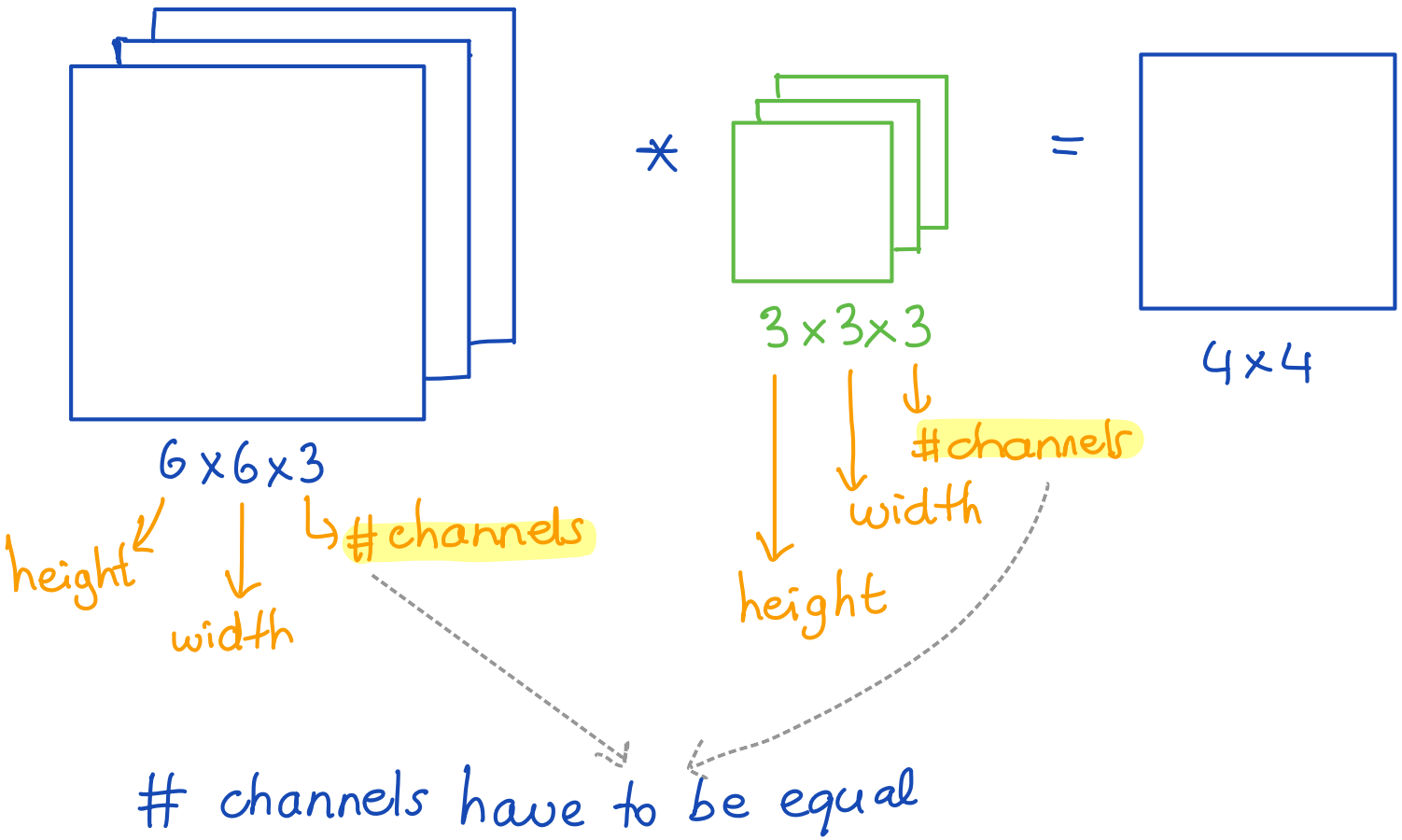
deeplearning.ai

Convolutional Neural Networks

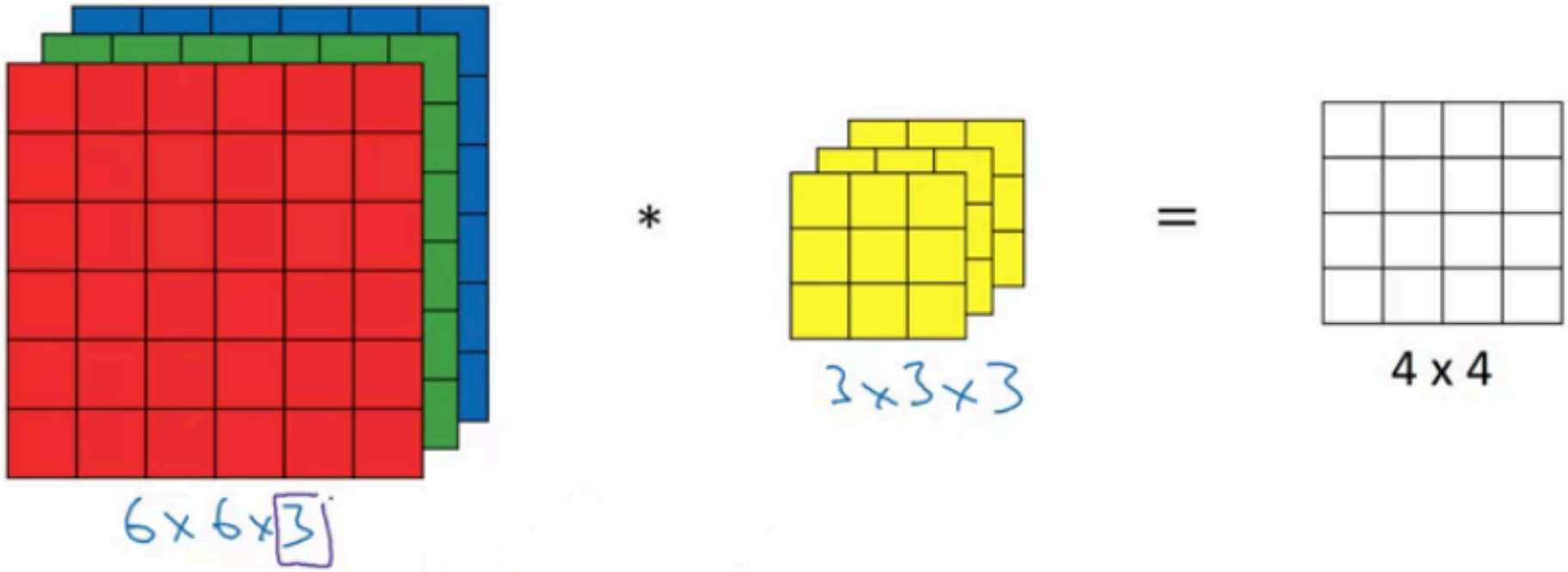
Convolutions over volumes

Convolutions on RGB images

- Goal: Detect features of an RGB image

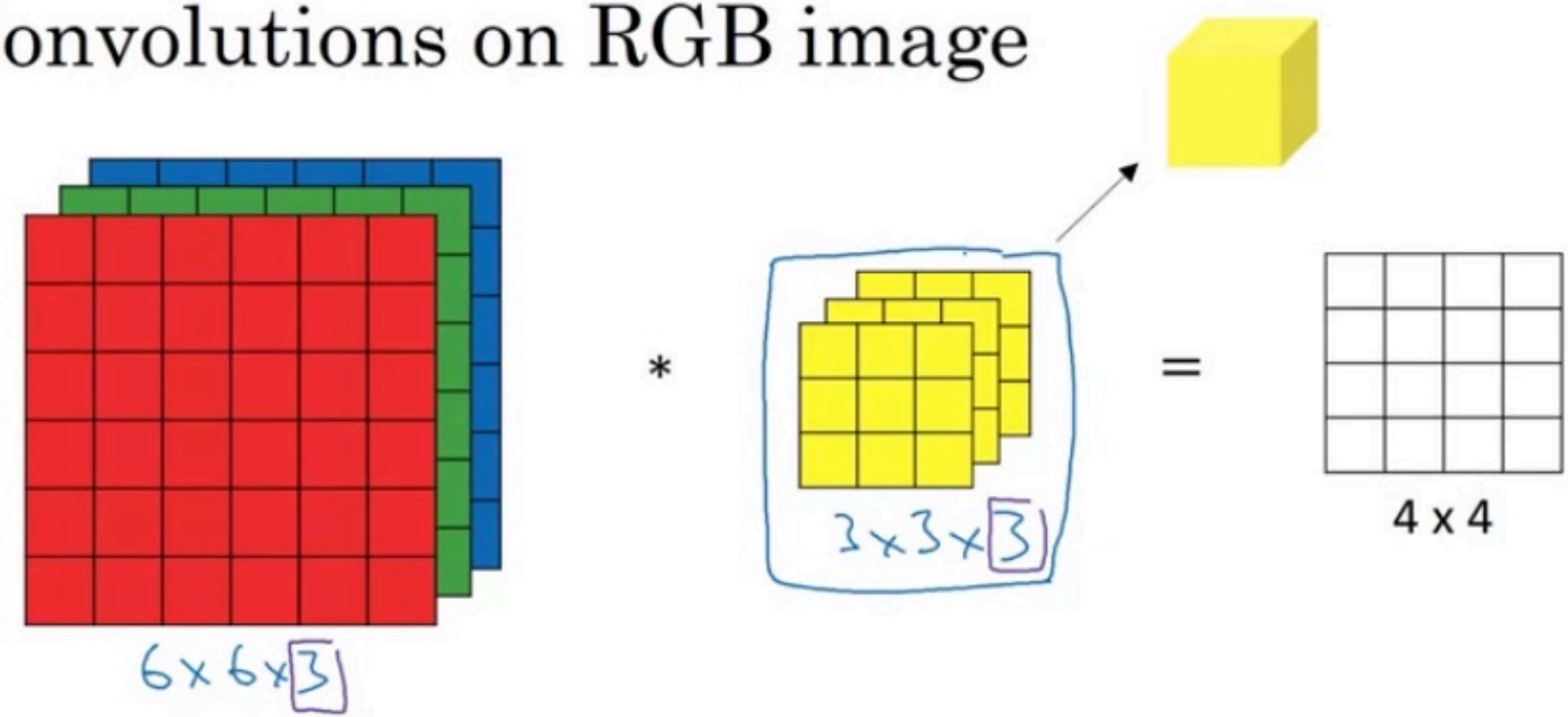


Convolutions on RGB image

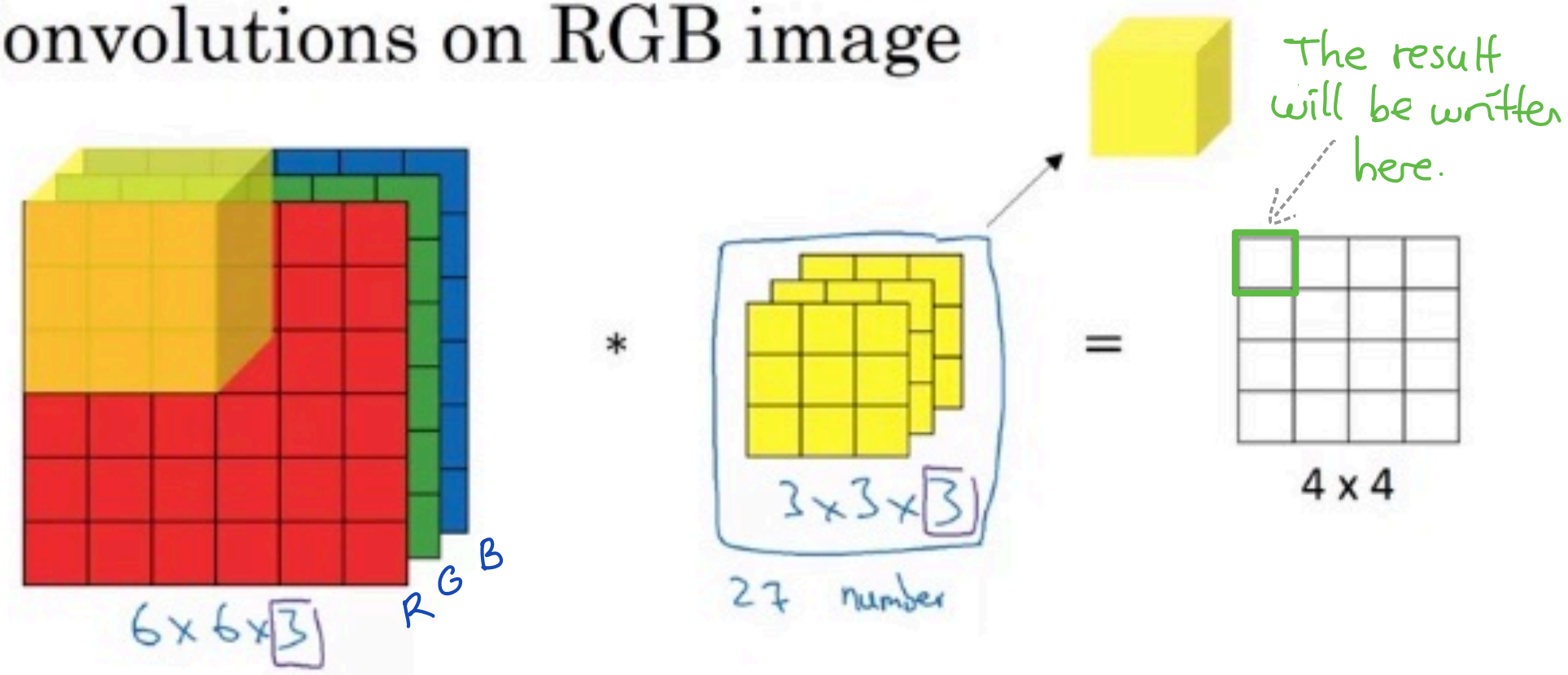


Andrew Ng

Convolutions on RGB image



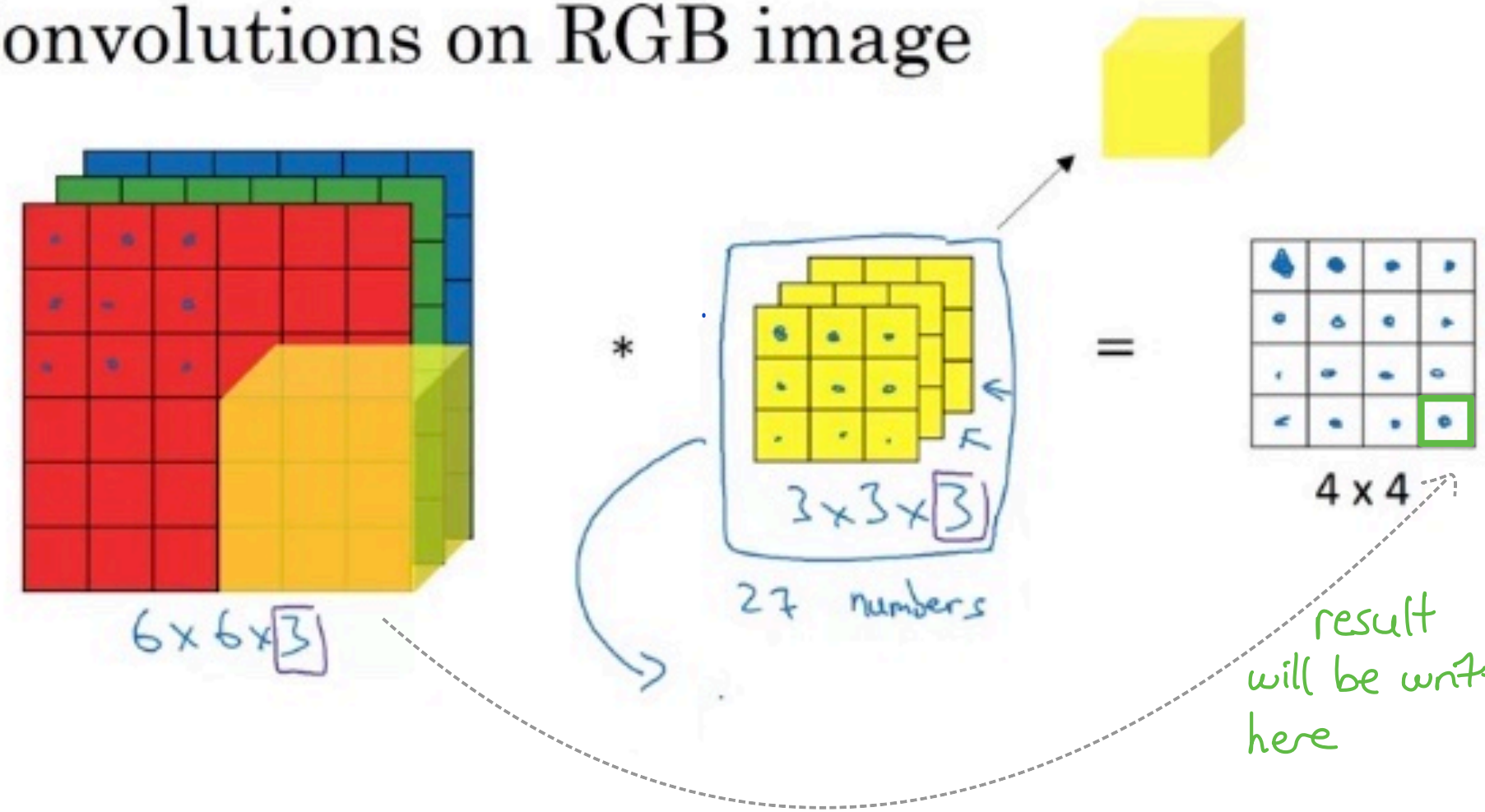
Convolutions on RGB image



- Take the 3x3 filter and place it in the upper left position. The filter has 27 parameters. Take the 9 numbers from the R, G, B channels and multiply with the corresponding filter coefficients.
- Then slide the filter by 1 pixel to the right and repeat.

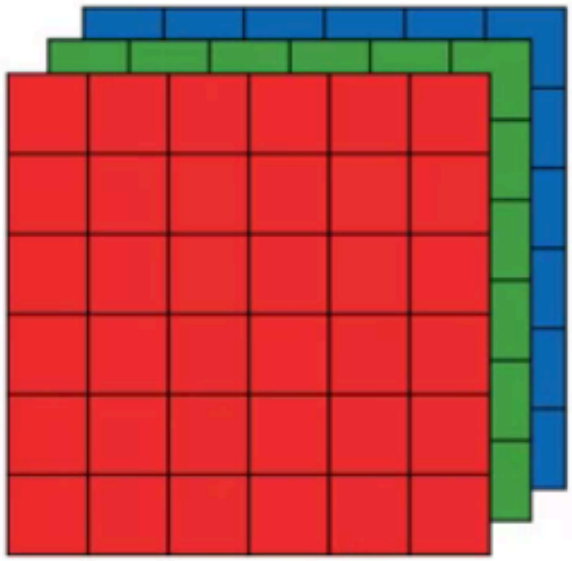
Andrew Ng

Convolutions on RGB image

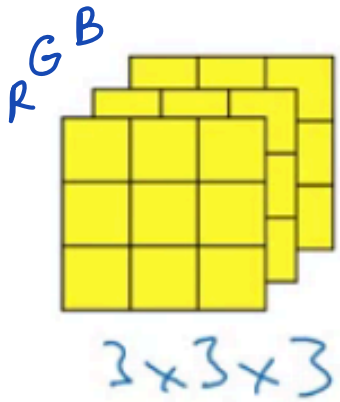


Andrew Ng

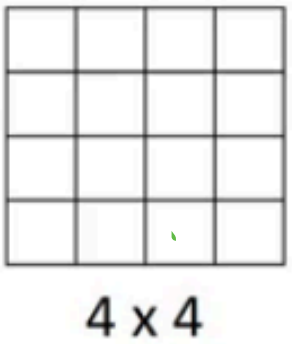
Convolutions on RGB image



*



=



Detect vertical edges in R channel:

R

1	0	-1
1	0	-1
1	0	-1

G

0	0	0
0	0	0
0	0	0

B

0	0	0
0	0	0
0	0	0

: 3x3x3

If the color of the vertical edge is not important :

1	0	-1
1	0	-1
1	0	-1

R

1	0	-1
1	0	-1
1	0	-1

G

1	0	-1
1	0	-1
1	0	-1

B

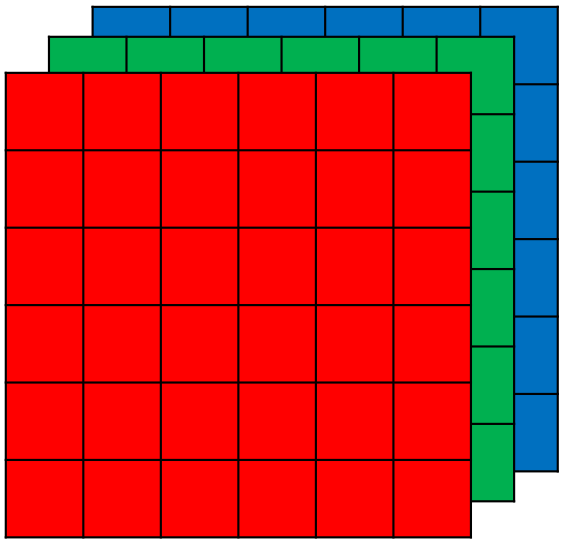
: 3x3x3

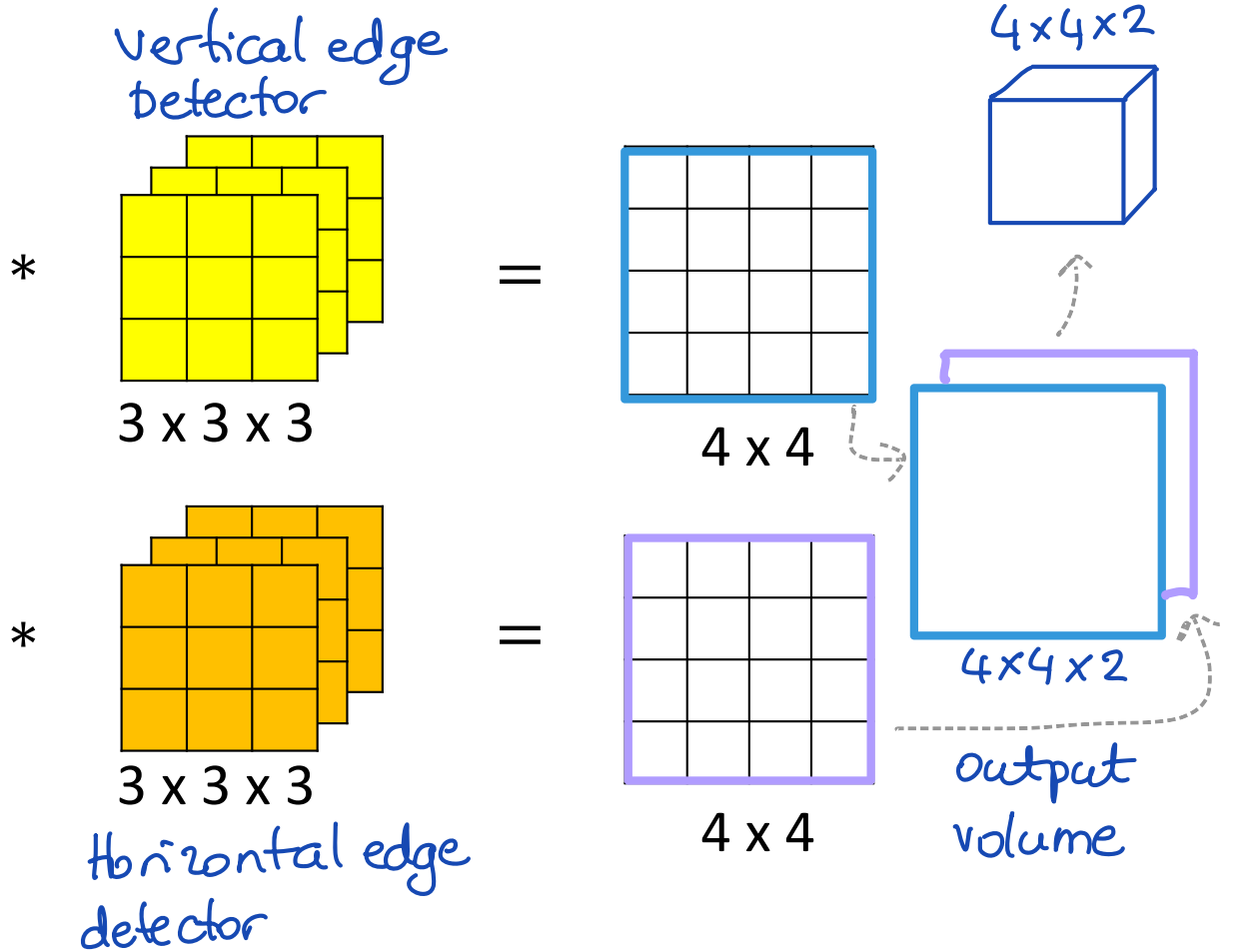
Andrew

- If we convolve a 6x6x3 image with a 3x3x3 filter, the output is 4x4.

- What if we don't want to detect vertical edges?
- What if we want to detect vertical edges, horizontal edges, edges of 45 degrees, 70° etc.?
- That is, we want to use multiple filters at the same time.

Multiple filters





Andrew Ng

Summary:

input	*	filter	⟶	output
$n \times n \times n_c$	*	$f \times f \times n_c$	⟶	$(n-f+1) \times (n-f+1) \times n_c$
$6 \times 6 \times 3$	*	$3 \times 3 \times 3$		$4 \times 4 \times 2$

stride of 1 and no padding

- We can now detect 2, 10 or maybe 128 or several hundreds of different features.

- $6 \times 6 \times 3$
  - ↳ # channels or "depth" of the 3D volume.



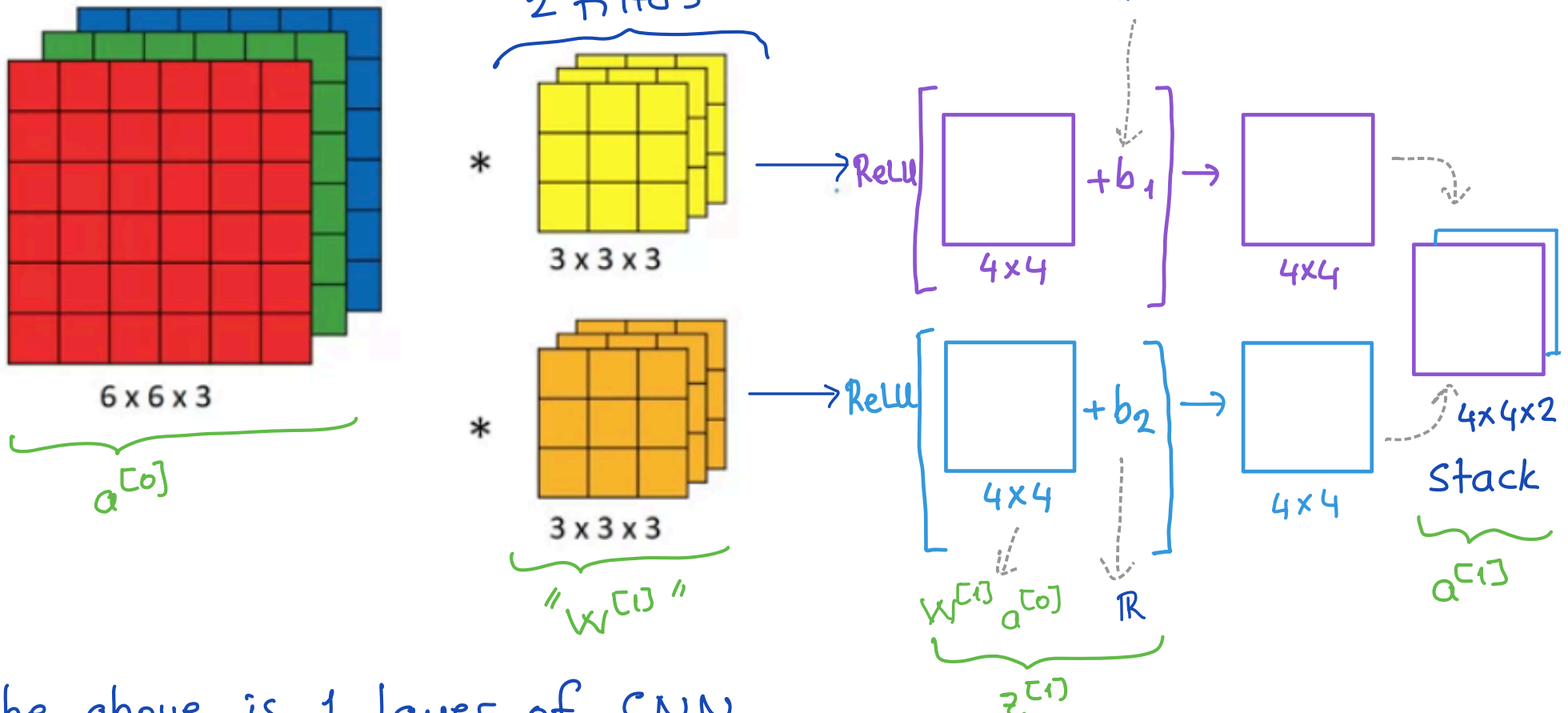
deeplearning.ai

Convolutional Neural Networks

One layer of a convolutional network

Goal: Build 1 layer of a CNN.

Example of a layer



- The above is 1 layer of CNN.

$$z^{[1]} = W^{[1]}\,a^{[0]} + b^{[1]}$$
$$a^{[1]} = g(z^{[1]})$$

} 1 step of forward propagation in a standard NN.

Andrew Ng

- To convert the filtering operation to a CNN layer, add a bias to all elements of filter outputs (16 elements) and apply a nonlinearity.
- Stack the results of the 2 filters.
- This is 1 layer of CNN.
- Let's map this to one layer of forward propagation in the standard neural network (a non-convolutional neural network)

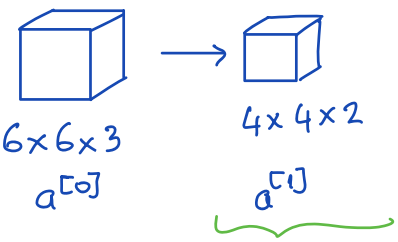
  One step of forward propagation:

  $$z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$$

  $$a^{[1]} = g(z^{[1]})$$

  filters $\leftrightarrow W^{[1]}$

  output of filters $\leftrightarrow W^{[1]} a^{[0]}$, since convolution is a linear operation.



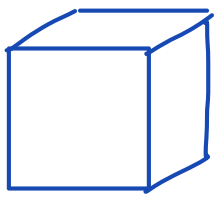
: 1 layer of CNN.

- If we had 10 filters → 4x4x10

Number of parameters in one layer

If you have 10 filters that are 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?

each filter:



$3 \times 3 \times 3 = 27$ params.

+ bias $= 28$ params

10 filters : $28 \times 10 = 280$ params.

Andrew Ng

NOTE: • No matter how big the input image is, the number of parameters remains fixed at 280.
The input image could be 1000×1000, 5000×5000 etc.
• You can use 10 filters to detect features (vertical edges, horizontal edges, other features) in a large image with a very small number of parameters.
• This makes CNNs less prone to over-fitting.

Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

→ Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

Activations: $a^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

bias: $n_c^{[l]} - (1,1,1,n_c^{[l]})$ ← #filters in layer l.

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$ ←

Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$ ←

$$n_{H,W}^{[l]} = \left\lfloor \frac{n_{H,W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$ → channels last notation

$n_c \times n_H \times n_W$ (channels first notation)

m: # training examples

Andrew Ng



deeplearning.ai

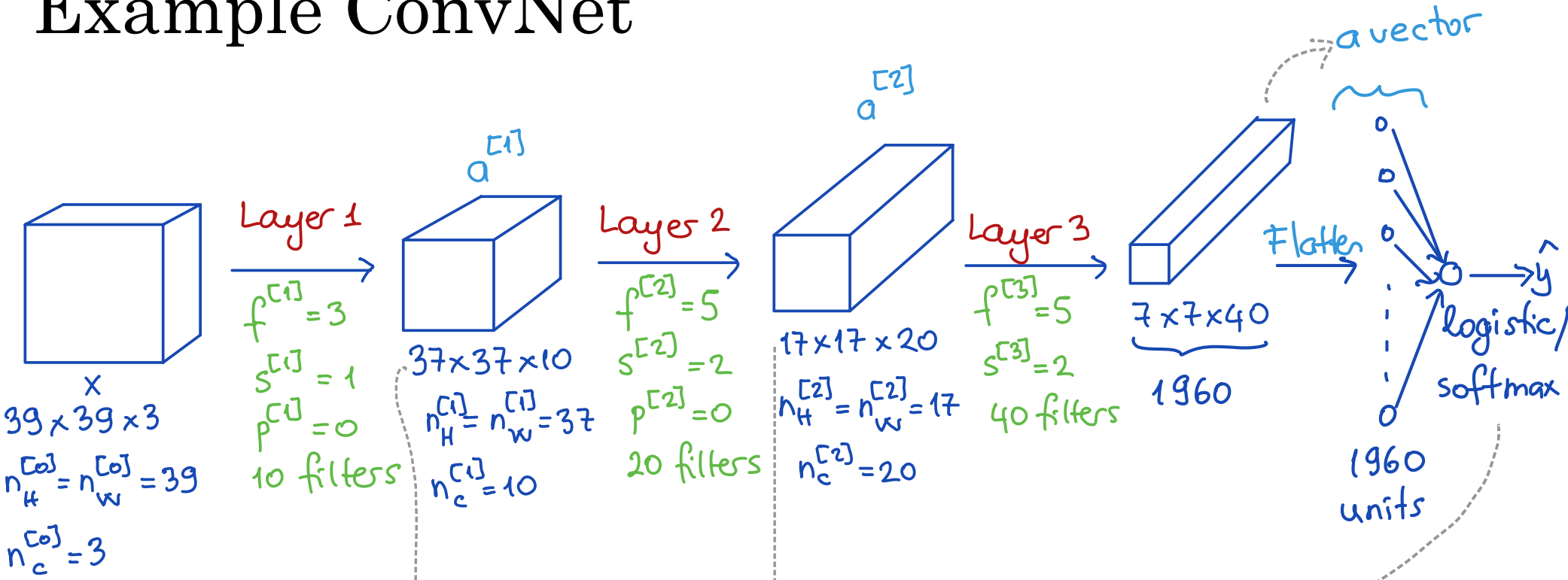
Convolutional Neural Networks

A simple convolution network example

(A Deep CNN example)

Goal: Build a CNN for this task.

Example ConvNet



$$\frac{n+2p-f}{s}+1$$

$$= \frac{39+0-3}{1}+1 = 37$$

$$\frac{n+2p-f}{s}+1$$

$$= \frac{37+0-5}{2}+1$$

$$= 17$$

logistic: cat / no cat

softmax: 3 or more classes

Andrew Ng

- A lot of the work in designing a CNN is selecting hyperparameters like filter size, stride, padding, number of filters used.

- As we go deeper in the network, the height and width of the activations will decrease, whereas the # channels will increase.

Types of layer in a convolutional network:

- Convolution (CONV)
- Pooling (POOL)
- Fully connected (FC)



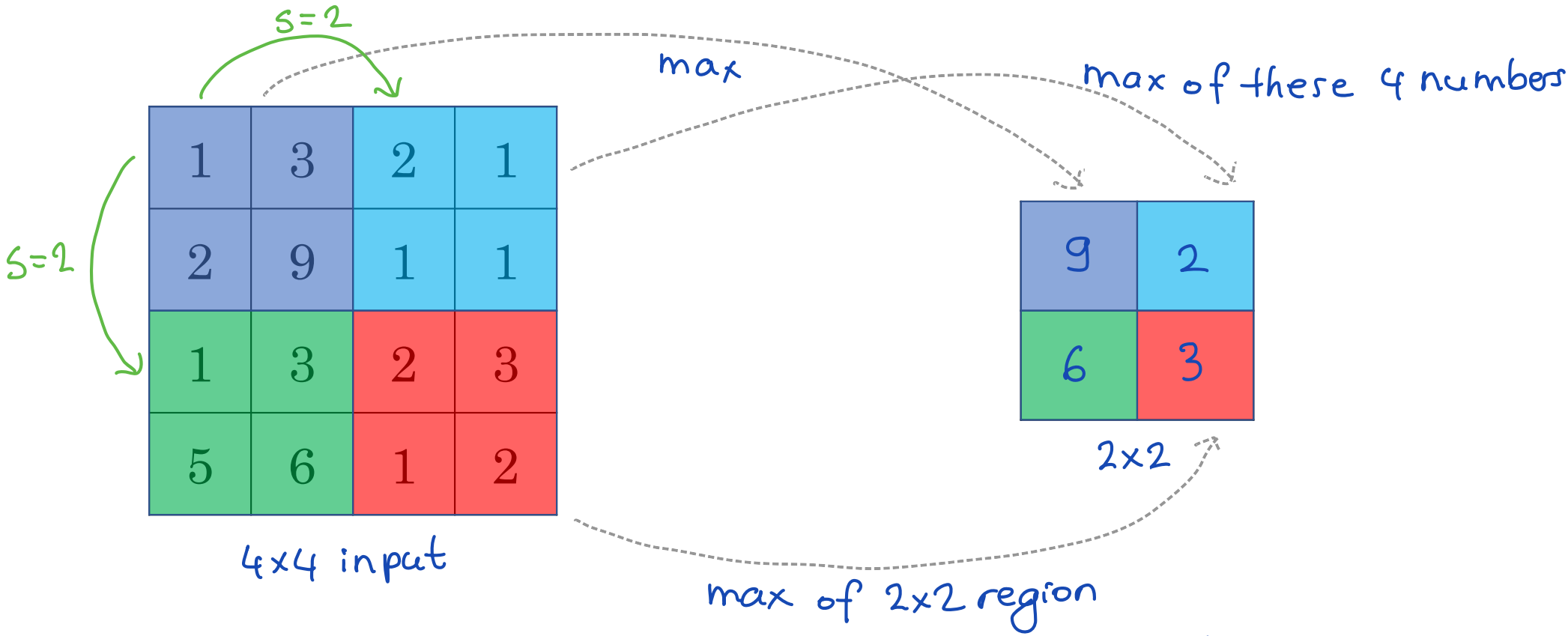
deeplearning.ai

Convolutional Neural Networks

Pooling layers

- Reduce the size of the representation.
- Speed up computation
- To make some of the feature detectors more robust.

Pooling layer: Max pooling



Hyperparameters of max-pooling

$f=2$ filter size (squares) is 2x2

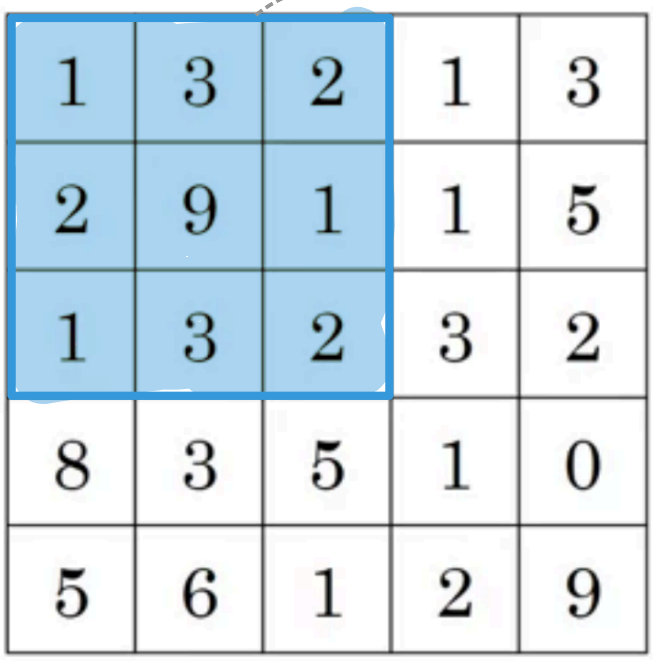
$s=2$ stride (step size) is 2

No parameters to learn during backpropagation.

Andrew Ng

- Intuition behind max-pooling:
  - The 4×4 region is some set of features, the activations of some layer of the neural network.
  - A large number means it detected a particular feature. (e.g. a vertical edge). That feature exists in the upper-left quadrant. It doesn't exist in the upper-right quadrant.
  - The max operation preserves the features (high numbers) detected anywhere.

Pooling layer: Max pooling



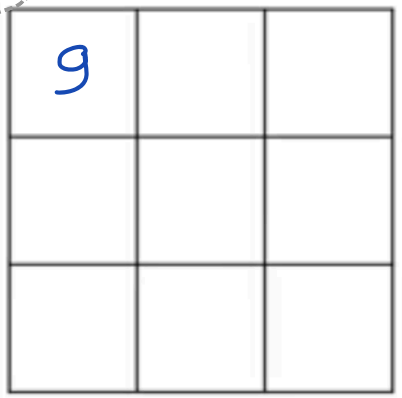
5×5

input

max

output

3×3



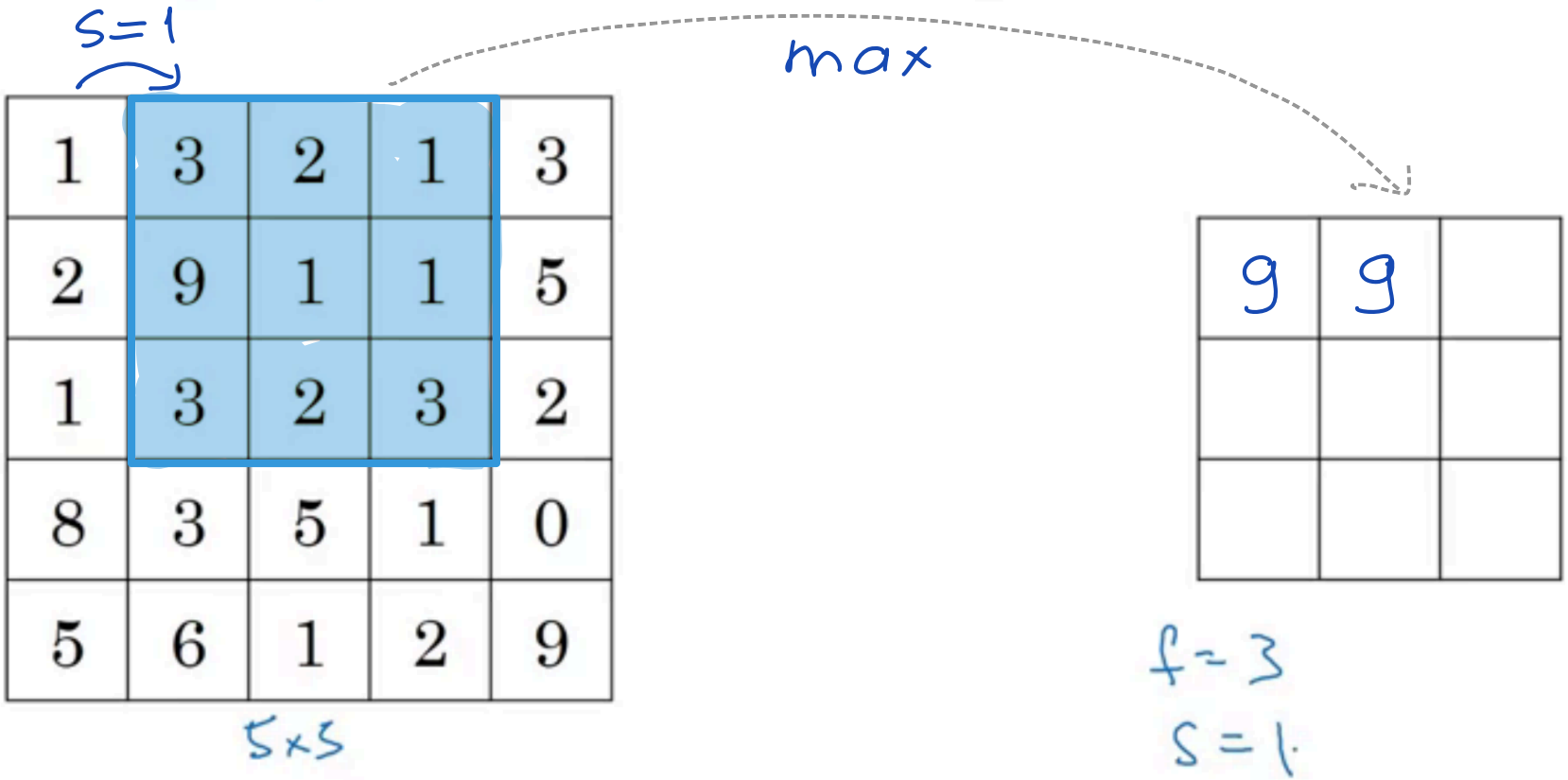
$f = 3$

$s = 1$

$$\text{output size} = \frac{n+2p-f}{s} + 1 = \frac{5-3}{1} + 1 = 3$$

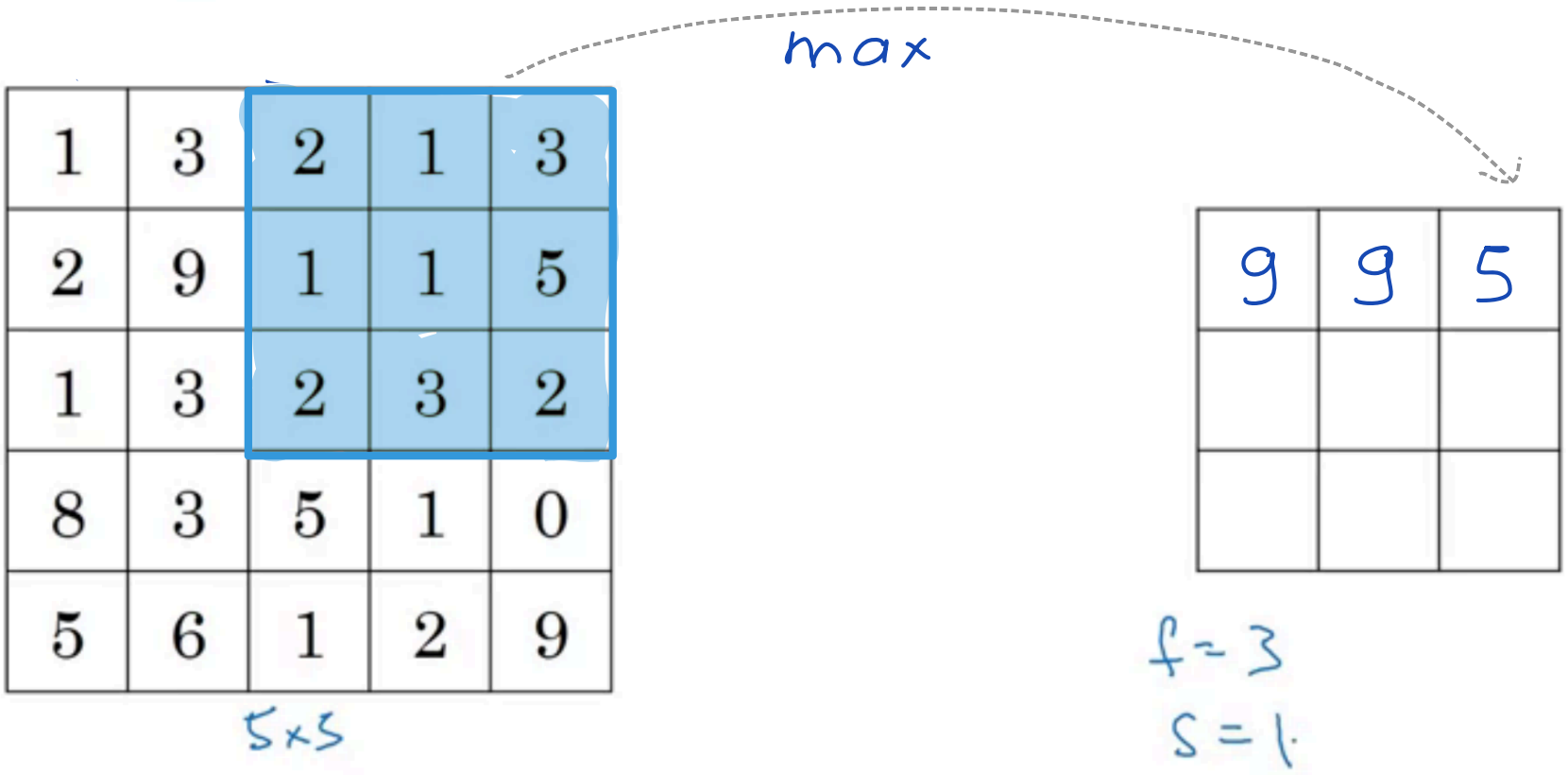
Andrew Ng

Pooling layer: Max pooling



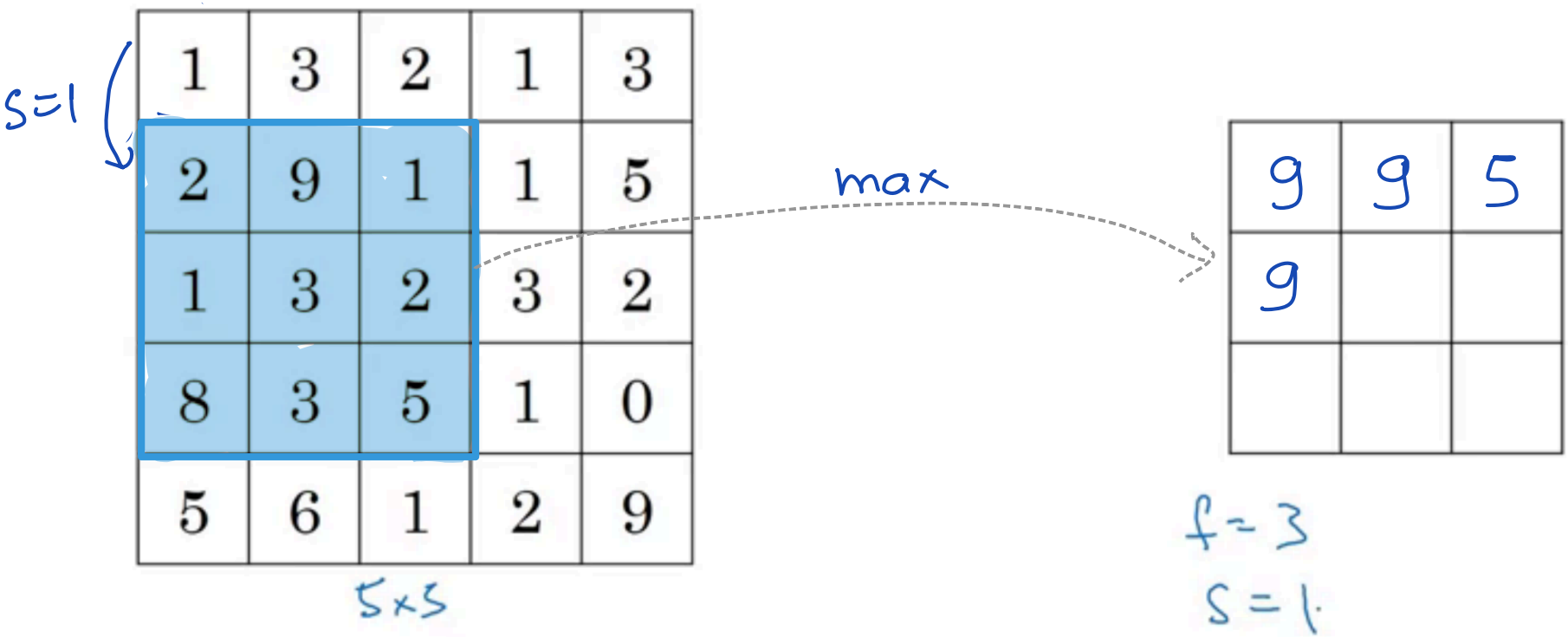
Andrew Ng

Pooling layer: Max pooling



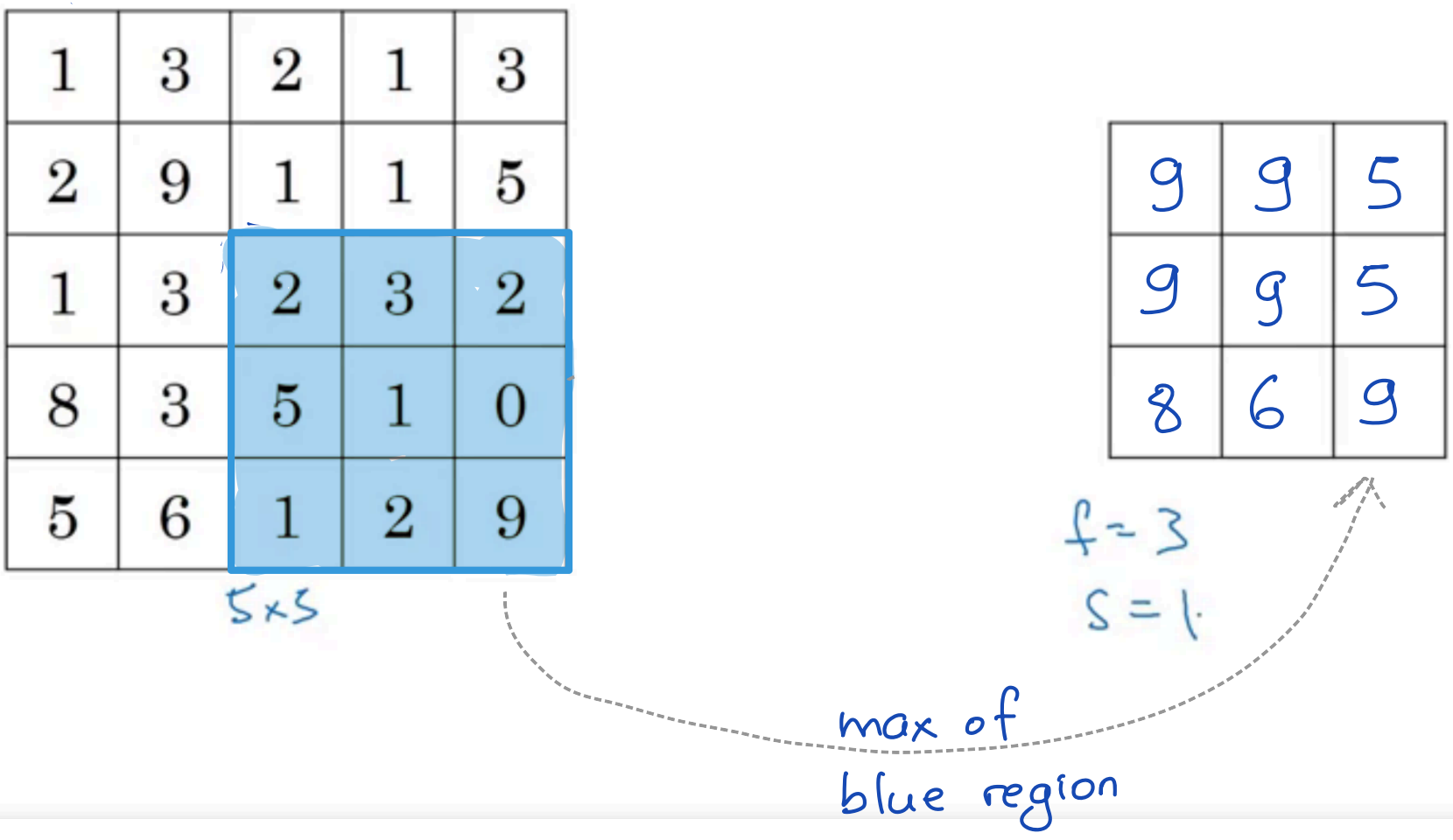
Andrew Ng

Pooling layer: Max pooling



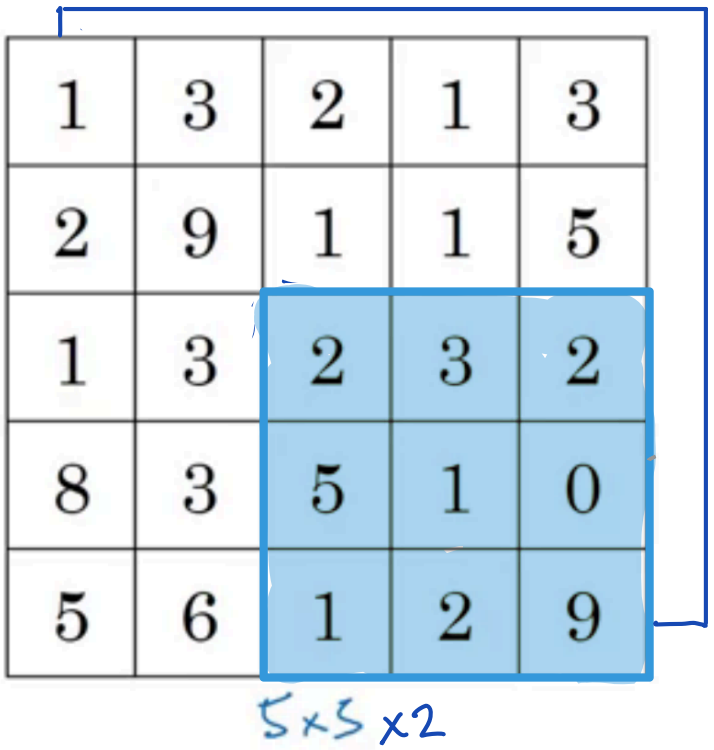
Andrew Ng

Pooling layer: Max pooling

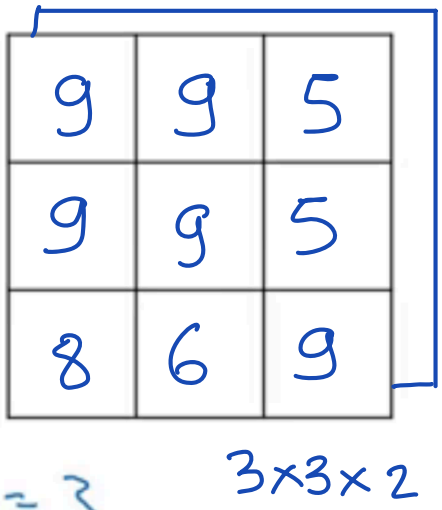


Andrew Ng

Pooling layer: Max pooling



→ if there are additional channels apply max pooling to them independently.



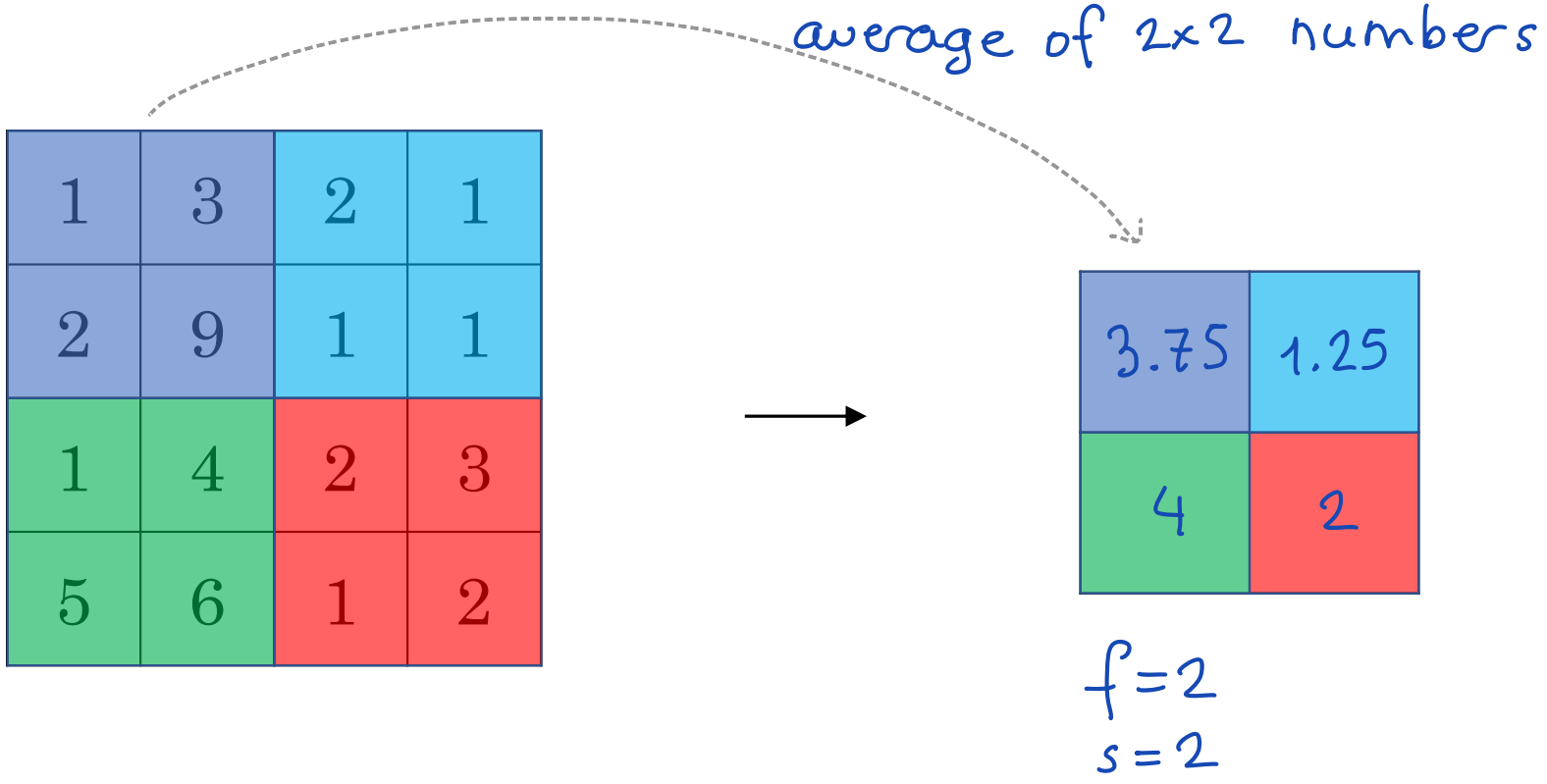
$f = 3$

$s = 1$

input $5 \times 5 \times n_c$ → max pooling → output $3 \times 3 \times n_c$

Andrew Ng

Pooling layer: Average pooling



- max pooling is used more often than average pooling.

Andrew Ng

Summary of pooling

Hyperparameters:

f : filter size (e.g. $f=2, s=2$ or $f=3, s=2$)

s : stride

Max or average pooling

Generally $p=0$ (no padding)

No parameters to learn!

Pooling is applied to each channel independently.

input size $n_H \times n_W \times n_c$

output size $\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \times n_c$

Andrew Ng



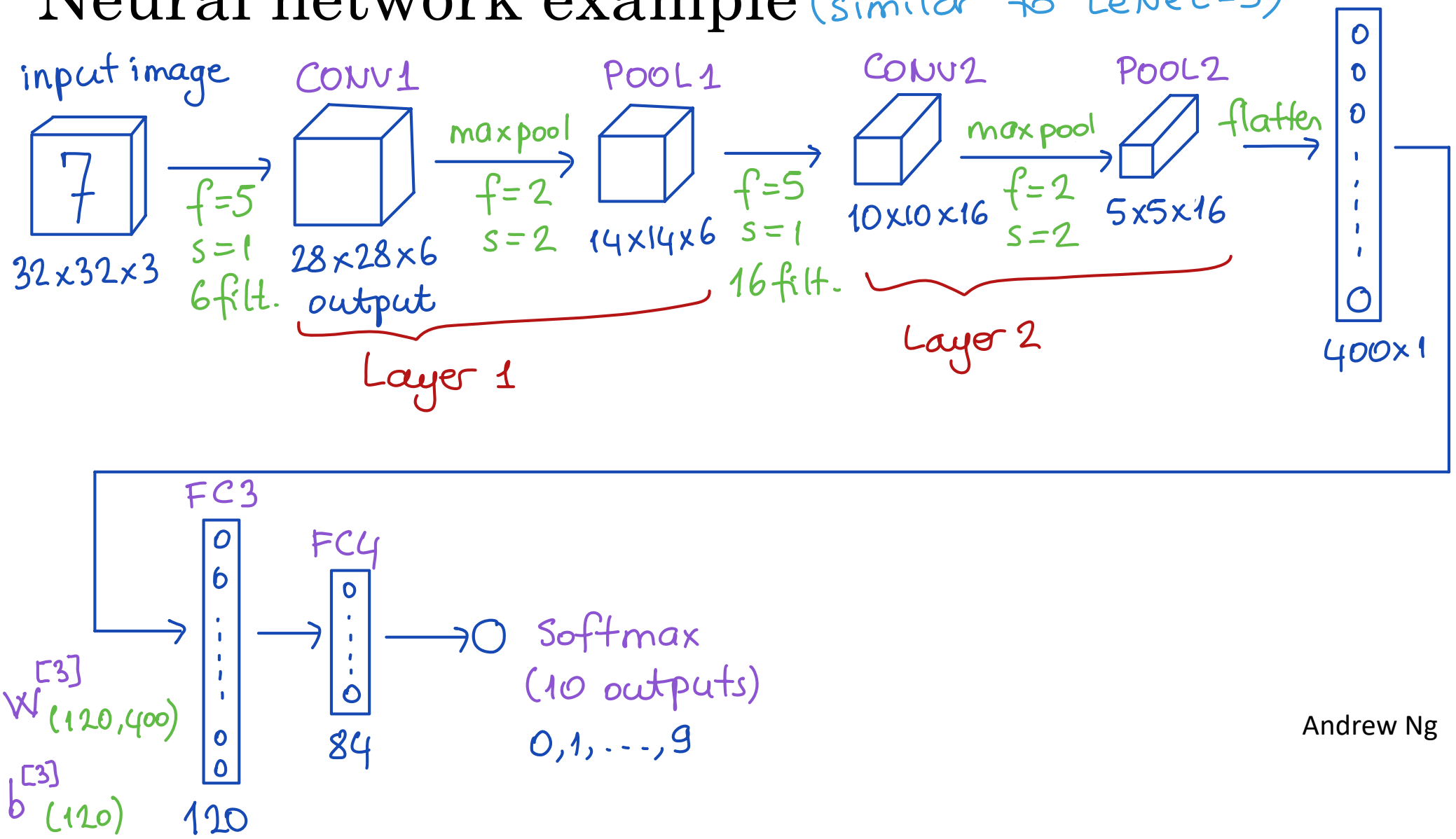
deeplearning.ai

Convolutional
Neural Networks

Convolutional neural network example

Goal: Recognize the handwritten digit in the RGB image.

Neural network example (similar to LeNet-5)



Andrew Ng

- Note: There are 2 conventions when counting the number of layers:
  1) Conv 1 + Pool 1 is counted as Layer 1
  2) Conv 1 is layer 1, Pool 1 is Layer 2

- In general, the layers that have weights are counted. Pooling layer has no weights (or parameters) to learn.

- FC: Fully Connected. Standard neural network.

- To select the hyperparameters see which values work well in the literature.

- As you go deeper in the network $n_H, n_W \downarrow$ and $n_C \uparrow$

- Another common pattern in CNNs:

  CONV – POOL – CONV – POOL – FC – FC – FC – SOFTMAX

Neural network example

	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072 $a^{[0]}$	0
CONV1 (f=5, s=1)	(28,28,6)	4707	456
POOL1	(14,14,6)	1176	0
CONV2 (f=5, s=1)	(10,10,16)	1600	2416
POOL2	(5,5,16)	400	0
FC3	(120,1)	120	48120
FC4	(84,1)	84	10164
Softmax	(10,1)	10	850

Andrew Ng

Observations about the table:

- Max pooling layers don't have any parameters
- Conv layers tend to have relatively few parameters.

CONV1, 456 params: $f=5$, 6 filters $\Rightarrow 5\times5\times3\times6 = 450 + 6 = 456$ params
(3 = #channels, 6 = bias)

CONV2, 2416 params: $f=5$, 16 filters, 6 channels
$5\times5\times6\times16 = 2400$, $2400 + 16$ bias $= 2416$ params

- Most of the parameters are in FC layers (48120, 10164)
- Activation size decreases as we go deeper in the network.



deeplearning.ai

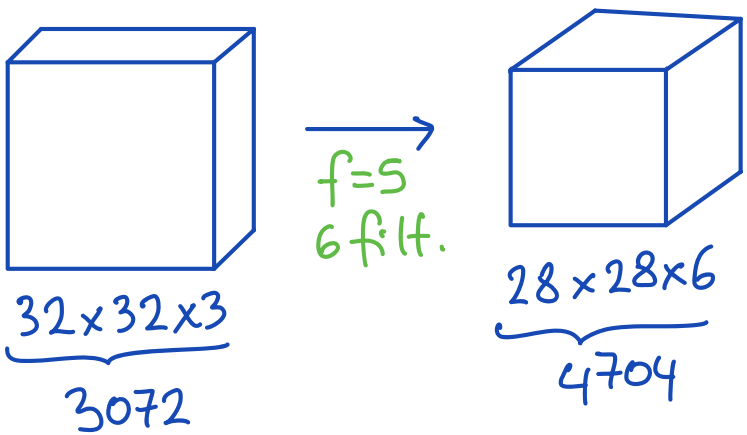
Convolutional Neural Networks

Why convolutions?

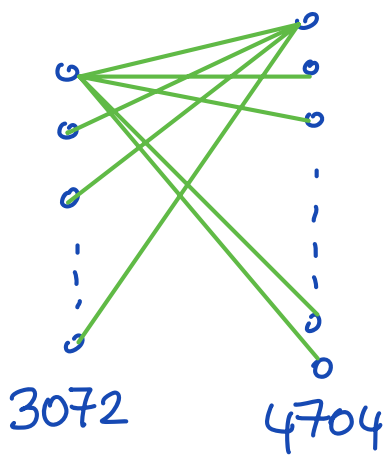
- Why are convolutions so useful when you include them in your NN?
- How to train a CNN when you have a labeled training set.

Why convolutions

Advantages: 1) parameter sharing 2) sparsity of connections.



- If you build a NN with 3072 units one layer and 4704 units in the next layer, the weight matrix would be $3072 \times 4704 \cong 14$ million.
  - A lot of parameters to train for a small image. (32x32)



Andrew Ng

- Using CNNs: $f=5$, 6 filters, the # of parameters are:

$$(5\times5\times3+1)\times6 = 456 \quad \text{(quite small)}$$

↳ bias

This is based on the equation:

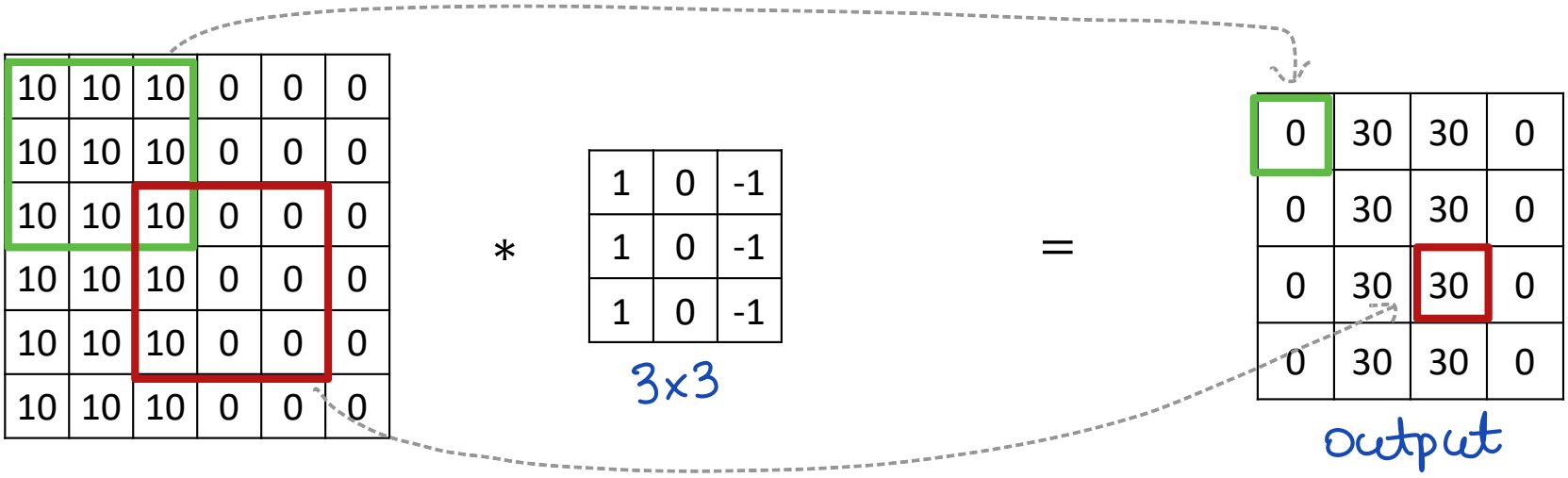
$$\left(f^{[l]} \times f^{[l]} \times n_c^{[l-1]} + 1\right) \times n_c^{[l]}$$

$f^{[l]}$: filter height and width

$n_c^{[l-1]}$: # channels in the previous layer.

$n_c^{[l]}$: # channels in the current layer.

Why convolutions



Parameter sharing: A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image. (translation invariance)

3x3 filter (9 parameters) is used to compute 4x4=16 outputs.

Sparsity of connections: In each layer, each output value depends only on a small number of inputs.

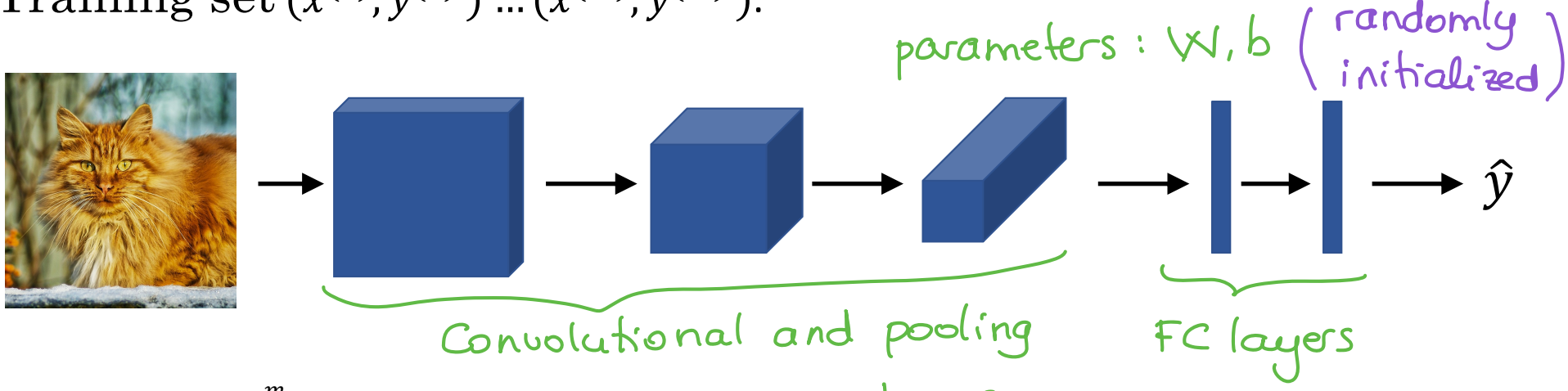
(The top left zero in the output is computed from only the 3x3 square of pixels on the left.)

- Through these two mechanisms, the CNN has a lot fewer parameters, which allows it to be trained with smaller training sets and is less prone to overfitting.

- A CNN is very good at capturing translation invariance. A picture of a cat shifted a couple of pixels to the right is still a cat. Convolution helps the NN encode the fact that an image shifted a few pixels should result in similar features and should be assigned the same label.

Putting it together – cat detector example

Training set $(x^{(1)}, y^{(1)}) \ldots (x^{(m)}, y^{(m)})$.



Cost $J = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

Use gradient descent to optimize parameters to reduce J